Here is the complete main prompt with the new repository eligibility and idea-validation section included:

Act as a senior open-source maintainer designing a realistic repository-level engineering challenge from a GitHub repository.

You will be provided:

- A repository URL
- A target commit hash
- Optionally, an existing problem statement for reference

Your task is to analyze the repository structure, architecture, public APIs, documentation, development history, and test patterns, then generate a complete engineering challenge suitable for evaluating senior-level software engineers.

The generated challenge must feel like a real upstream engineering task from a mature production repository.

Challenge Requirements

General Requirements

- The task difficulty should require approximately 6–8 hours for a strong senior engineer.
- The implementation must require reasoning across multiple modules and abstractions.
- The challenge must be deterministic and fully offline-testable.
- No internet access should be required during test execution.
- The repository's relevant existing tests must pass on the base commit.
- The new or modified tests must fail on the base commit and pass after applying the solution patch.
- Tests must validate behavior strictly through the repository's public APIs.
- Avoid toy problems, isolated utilities, artificial algorithms, mechanical refactors, or trivial validation changes.
- The problem must be novel and non-obvious.
- Do not generate common benchmark-style tasks involving:
  - PR open, close, reopen, or reject flows
  - Simple CRUD operations
  - Basic authentication or authorization checks
  - Straightforward parser additions
  - Trivial configuration flags
  - Shallow cache invalidation logic
  - Simple pagination, filtering, or sorting behavior
- The task should require architectural understanding, state coordination, or cross-module consistency handling.

Implementation Requirements

- The solution patch must modify exactly 3–4 existing repository files.
- The implementation patch must contain at least 600 lines of meaningful production-quality code.
- Changes must be interconnected and necessary.
- No comments.
- No docstrings.
- No dead code.
- No placeholder implementations.
- Code must follow the repository's existing style, architecture, and engineering conventions.

Testing Requirements

- The test patch must contain only test-related changes, including the root-level "test.sh" runner.
- The test patch must contain at least 700 lines of code.
- Include at least 40 deterministic test cases.
- Tests must cover edge cases, integration behavior, regression scenarios, and API-level correctness.
- Tests must follow the repository's existing testing conventions and framework.
- Avoid superficial or repetitive assertions.
- Relevant existing tests must pass without the solution patch.
- New or modified tests must fail without the solution patch and pass after it is applied.
- Do not use fail-fast options.
- Every selected test must run so that the JUnit report contains all results.

Problem Description Requirements

- Write the description in natural, professional English with a smooth and logical flow.
- Keep it at most 250 words while avoiding filler, repetition, or unnecessary wording.
- Ensure all information remains accurate and consistent with the source material, without adding, removing, or altering any technical details.
- Verify that there are no mismatches, contradictions, or inconsistencies between the description and any referenced requirements, specifications, tests, or results.
- Improve clarity and readability while preserving the original meaning and intent.

Repository Eligibility and Idea Validation Requirements

Before proposing a challenge title or problem description, verify that the repository and feature idea satisfy all of the following requirements.

R1 — Repository Activity

- Do not use an inactive, archived, abandoned, or effectively unmaintained repository.
- Confirm that the repository has meaningful recent development activity.
- Review recent commits, releases, issue responses, merged pull requests, and maintainer participation.
- Do not rely only on star count, repository age, or the presence of occasional automated dependency updates.
- If the repository does not appear actively maintained, do not use it.

R2 — No Existing Pull Request

- Search all open, merged, and closed pull requests for implementations substantially equivalent to the proposed feature.
- Reject the idea if any existing pull request already implements, substantially attempts, or meaningfully overlaps with the proposed behavior.
- A closed, abandoned, rejected, draft, or unmerged pull request still makes the idea ineligible.
- Do not treat failure to merge as evidence that the idea is fresh.
- This requirement applies even when the existing pull request uses a different implementation approach, title, terminology, or API shape.
- Search using behavioral concepts, likely technical terms, related API names, and synonyms rather than relying on one exact phrase.

R3 — No Maintainer Rejection

- Search open and closed issues and GitHub Discussions for prior proposals, design decisions, maintainer feedback, rejected requests, and architectural rulings related to the idea.
- Do not limit the search to pull requests.
- Reject the idea if maintainers have previously declined, ruled against, discouraged, or explicitly decided not to support the proposed behavior.
- A declined feature counts as misalignment even when no implementation was submitted.
- Do not propose a feature that conflicts with established design decisions, compatibility guarantees, maintenance priorities, or architectural direction.

R4 — Project Fit

- Read the README and relevant project documentation before selecting the task.
- Review contribution guidelines, architecture documentation, design documents, roadmaps, release notes, examples, and supported-use-case documentation when available.
- Understand the project’s domain, philosophy, scope, abstractions, and intended user experience.
- Ensure the feature fits naturally within the project and could reasonably be accepted as an upstream contribution.
- Do not invent nonsensical, artificial, benchmark-oriented, out-of-scope, or philosophically incompatible behavior merely to satisfy complexity, file-count, test-count, or line-count requirements.

- Prefer a genuine missing capability, unresolved consistency problem, lifecycle gap, protocol requirement, or production-relevant behavior.

Validation Summary

- During the proposal stage, briefly confirm that:
  - The repository is active.
  - No open, merged, or closed pull request already implements or substantially attempts the idea.
  - No issue or GitHub Discussion records a maintainer decision declining the idea.
  - The feature aligns with the project's documented scope and philosophy.
- Base these confirmations on actual repository evidence.
- Do not claim that a check passed when it could not be verified.
- If any eligibility check fails or cannot be verified with reasonable confidence, do not propose that task.
- Select another feature idea or state that the supplied repository is unsuitable.

Approval Gate Before File Generation

Before generating "problemtitle.txt", "problem_description.txt", "test.patch", "solution.patch", "test.sh", or the Dockerfile:

- Analyze the repository, target commit, architecture, public APIs, documentation, maintenance activity, pull requests, issues, GitHub Discussions, and existing test patterns.
- Complete all Repository Eligibility and Idea Validation Requirements.
- Present only:
  - "Proposed Title"
  - "Proposed Problem Description"
  - "Eligibility Validation"
- The proposed title must be concise, specific, and suitable for "problemtitle.txt".
- The proposed description must satisfy all Problem Description Requirements.
- The description must specify externally observable behavior without exposing exact files, functions, classes, algorithms, or required internal architecture changes.
- The eligibility validation must briefly confirm the four repository and idea checks without exposing unnecessary research notes.
- Verify that the proposed task is realistic for the repository.
- Verify that the task can naturally support:
  - A 6–8 hour senior-level implementation
  - At least 600 lines of meaningful interconnected production code
  - Changes to exactly 3–4 existing production files
  - At least 40 meaningful deterministic test cases
  - At least 700 lines of genuine test code
- Do not generate or provide any patch, test script, implementation, Dockerfile, or final output file during this initial stage.

- After presenting the proposed title, problem description, and eligibility validation, stop.
- Generate the remaining files only when the user explicitly says "proceed".
- Do not interpret questions, comments, requested edits, partial approval, or similar responses as permission to proceed.
- When the user requests changes, revise the proposed title or description as requested and stop again.
- Wait for the exact instruction "proceed" before generating the files.
- After the user says "proceed", preserve the approved title and description exactly unless the user explicitly requests further changes.

Generated Files

After the user explicitly says "proceed", produce exactly these files in the following order:

1. "problemtitle.txt"

- Contains the approved challenge title on a single line.

2. "problem_description.txt"

- Contains the approved problem description.
- Must satisfy all Problem Description Requirements.

3. "test.patch"

- Valid unified Git patch.
- Must contain only test-related modifications.
- Must add "test.sh" at the repository root.
- Must include all new or modified tests.

4. "solution.patch"

- Valid unified Git patch.
- Must contain only production implementation changes.

5. "Dockerfile"

- Must satisfy all Dockerfile Requirements below.

Do not include explanatory text outside these files during the final generation stage.

Test Script Requirements

Create a "test.sh" file at the repository root and include it in "test.patch".

The script must support exactly these two modes:

"./test.sh --output_path results.xml base"

- Runs the repository's relevant existing tests as a regression check.
- Must pass before and after applying "solution.patch".
- Must write a valid JUnit XML report to the supplied output path.
- Must run the real existing tests for the area affected by the implementation, not a token smoke test.

"./test.sh --output_path results.xml new"

- Runs only the new or modified tests introduced by "test.patch".
- Must fail when only "test.patch" is applied.
- Must pass after both "test.patch" and "solution.patch" are applied.
- Must write a valid JUnit XML report to the supplied output path.

Additional Test Script Requirements

- Correctly parse "--output_path", its value, and the "base" or "new" mode.
- Create the output directory when necessary.
- Return the underlying test command's exit status.
- Reject missing, invalid, unexpected, or duplicate arguments with a non-zero exit status.
- Do not use fail-fast flags or configurations.
- Ensure all selected tests execute and appear in the JUnit report.
- Keep the script deterministic and compatible with offline execution.
- It is acceptable to exclude existing tests that are genuinely flaky, require network access, or already fail for unrelated reasons.
- Do not exclude valid regression tests because the solution causes them to fail.
- Keep exclusions narrow and limited to demonstrably unrelated failures.

JUnit XML Setup by Framework

Use the repository's existing test framework and an appropriate JUnit-compatible command.

pytest — built-in:

"pytest tests/ -v --junitxml="$OUTPUT_PATH""

Vitest — built-in reporter:

"vitest run tests/ --reporter=junit --outputFile="$OUTPUT_PATH""

Jest — "jest-junit" is pre-installed in the base image:

"JEST_JUNIT_OUTPUT_DIR="$(dirname "$OUTPUT_PATH")" \
JEST_JUNIT_OUTPUT_NAME="$(basename "$OUTPUT_PATH")" \ npx jest tests/
--reporters=jest-junit"

Go — "go-junit-report" is pre-installed in the base image:

"go test -v -count=1 ./pkg/... -timeout 10m 2>&1 \ | go-junit-report -set-exit-code >
"$OUTPUT_PATH""

The "-v" flag is required because "go-junit-report" parses verbose output.

Mocha — "mocha-junit-reporter" is pre-installed in the base image:

"mocha tests/ --timeout 10000 \ --reporter mocha-junit-reporter \ --reporter-options
mochaFile="$OUTPUT_PATH""

Deno — built-in:

"deno test tests/ --junit-path="$OUTPUT_PATH""

Adapt the paths and selectors to the repository so that:

- "base" runs the relevant existing regression suite.
- "new" runs only the tests introduced or modified by "test.patch".

If the repository uses another test framework:

- Use the repository's existing test framework rather than replacing it.
- Configure or install an appropriate JUnit XML reporter during the Docker build.
- Ensure the reporter works without network access at runtime.
- Ensure the command preserves the real test-process exit status.

Generating and Validating the Test Patch

Make the test changes, including "test.sh", and then generate the patch:

"git diff > test.patch"

Verify that the patch applies cleanly:

"git stash"
"git apply test.patch"

"git stash pop"

Ensure that:

- "test.patch" includes "test.sh" and all test changes.
- "test.patch" contains no production implementation changes.
- "test.patch" applies cleanly to the specified target commit using "git apply".

Patch Validation Requirements

Before returning the files, verify all of the following:

- "test.patch" applies cleanly to the target commit.
- "solution.patch" applies cleanly after "test.patch".
- "solution.patch" also applies cleanly to the target commit independently unless the implementation genuinely depends on test-only fixture files.
- The "base" test mode passes with only "test.patch" applied.
- The "new" test mode fails with only "test.patch" applied.
- The "base" test mode passes after both patches are applied.
- The "new" test mode passes after both patches are applied.
- The Docker image builds without either patch applied.
- Tests run successfully with networking disabled at runtime.
- The generated JUnit XML files are valid and contain all selected test results.
- No production changes appear in "test.patch".
- No test-only changes appear in "solution.patch".
- Exactly 3–4 existing production files are modified by "solution.patch".
- The production implementation contains at least 600 meaningful added or substantially modified lines.
- The test patch contains at least 700 meaningful lines and at least 40 behaviorally distinct deterministic test cases.

Patch Hygiene

Treat every added or modified file as if it were part of a genuine upstream pull request.

- Do not create files or directories containing "challenge", "quest", or "olympus" in their names.
- Do not include comments or text in "test.sh", tests, patches, or implementation files that refer to the evaluation task.
- Do not include "Shipd", "Olympus", or "mars" anywhere inside "test.patch" or "solution.patch".
- Do not expose benchmark instructions, hidden evaluation details, or artificial task terminology in repository files.
- Every filename, test name, fixture, helper, and code path must fit naturally within the repository.
- Do not manipulate test counts through generated duplicates, aliases, meaningless parameter combinations, or repetitive assertions.

- Do not add code solely to satisfy line-count requirements.
- Do not weaken existing assertions, skip relevant tests, or modify expected results merely to make the solution pass.
- Do not modify repository metadata, contribution files, or unrelated configuration unless strictly necessary for test execution.

Dockerfile Requirements

We need consistent results, so you must submit a Dockerfile that will be used to run the agent and tests inside.

It must follow these requirements:

- Start "FROM" one of the approved base images below.
- Select the image matching the repository's primary implementation language.
- Install all required dependencies during the build phase.
- The container will run with "--network none", so everything required must be installed and configured at build time.
- "WORKDIR" must be "/app".
- Tests will run from "/app"; other paths may break imports, editable installations, and relative paths.
- Copy the repository contents into "/app".
- Do not clone the repository inside the Dockerfile.
- Do not apply "test.patch" or "solution.patch" during the Docker build.
- Do not run tests during the image build.
- Do not include test commands in any "RUN" step.
- The Dockerfile must build and work from the unpatched repository state.
- End with "CMD ["/bin/bash"]".

Base Images

Select the image matching the repository's primary language:

Python
"public.ecr.aws/d3j8x8q7/olympus-base-python:latest"

TypeScript / JavaScript
"public.ecr.aws/d3j8x8q7/olympus-base-typescript:latest"

Go
"public.ecr.aws/d3j8x8q7/olympus-base-go:latest"

Rust
"public.ecr.aws/d3j8x8q7/olympus-base-rust:latest"

Java (JVM)
"public.ecr.aws/d3j8x8q7/olympus-base-jvm:latest"

C++
"public.ecr.aws/d3j8x8q7/olympus-base-cpp:latest"

Dockerfile Template

Use the following skeleton as a starting point. Replace "<language>" with "python", "typescript", "go", "rust", "jvm", or "cpp":

FROM public.ecr.aws/d3j8x8q7/olympus-base-<language>:latest

WORKDIR /app

COPY . .

Install everything required here. The runtime container is offline.

Example for Python:

RUN pip install -r requirements.txt

CMD ["/bin/bash"]

Additional Constraints

- Ensure the generated feature integrates naturally with the repository domain.
- Prefer changes involving synchronization semantics, transactional consistency, incremental computation, event ordering, serialization invariants, protocol correctness, lifecycle coordination, dependency propagation, streaming behavior, retry semantics, scheduler behavior, or multi-layer state management.
- Ensure tests genuinely exercise implementation complexity rather than artificially increasing line count.
- Generated patches must apply cleanly using "git apply".
- Do not generate intentionally bloated code or meaningless line inflation.
- Every added line in both patches must contribute to actual functionality or verification logic.
- Resolve any conflict between the requested constraints and the repository's actual architecture before proposing the title and description.
- Do not silently weaken, ignore, or reinterpret a requirement.
- If a requirement is genuinely incompatible with the repository, state the incompatibility during the proposal stage instead of generating invalid patches.